

[Low] Global UUID map in `blaze-v1` lets anyone pre-consume a victim's intent UUID (griefing DoS across all subnets)

Project: Charisma / Blaze Protocol

Contract (deployed, mainnet): `SP2ZNGJ85ENDY6QRHQ5P2D4FXKGZWCKTB2T0Z55KS.blaze-v1`

Author: rozar.btc — repo `github.com/r0zar/charisma` (`packages/clarity/contracts`)

Severity (self-assessed): Low — griefing / denial-of-service, no theft and no permanent loss of funds. The victim recovers by re-signing with a fresh UUID. Included because it affects every subnet built on `blaze-v1` (one shared replay map) and the fix is small.

Disclosure: Good-faith private disclosure. No mainnet exploitation was performed — the accompanying verifier is read-only (public API reads only, no transaction). No public disclosure.

Summary

`blaze-v1` is the SIP-018 intent verifier and replay-guard underneath all Blaze subnets. Its replay map, `submitted-uuids`, is **global and keyed on the UUID string alone** — not on the signer, and not on the calling contract. `execute` consumes the UUID with `map-insert` as its first action, then verifies the signature.

Because the map key carries no notion of **who** an intent belongs to, any account can consume any UUID by calling `execute` directly with an intent they signed themselves that happens to reuse the target UUID. Once consumed, the legitimate owner's intent bearing that same UUID reverts with `ERR_UUID_SUBMITTED` on every subnet, since they all share this one map. The attacker needs only to learn the victim's UUID before it lands on-chain — which is the normal condition in a broadcast-intent / solver model, where signed intents (UUID included) circulate off-chain or sit in the mempool before submission.

Vulnerability details

Live source of `blaze-v1` (fetched from the Hiro API; identical to the repo):

```
(define-map submitted-uuids (string-ascii 36) bool)

(define-public (execute
  (signature (buff 65))
  (intent    (string-ascii 32))
  (opcode    (optional (buff 16)))
  (amount    (optional uint))
  (target    (optional principal))
  (uuid      (string-ascii 36))
)
  (if (map-insert submitted-uuids uuid true)
    (verify (try! (hash contract-caller intent opcode amount target uuid)) signature)
    ERR_UUID_SUBMITTED
  )
)
```

- `map-insert` returns `true` only if `uuid` was **not** already present; on a repeat it returns `false` and `execute` yields `ERR_UUID_SUBMITTED`. So the first party to insert a given UUID wins it, permanently, for the whole protocol.
- The map key is only the `(string-ascii 36)` UUID. The signer (recovered inside `verify`) and the `contract-caller` are folded into the **hash that must be signed**, but **not** into the **replay key**. So two entirely unrelated intents that merely share a UUID collide.
- A failed signature does **not** burn the UUID: `execute` returns the `(err ...)` from `verify`, the public-function call rolls back, and the `map-insert` is undone. This correctly prevents a **free** griefing burn — but it does not prevent the attack, because the attacker can supply a **valid** signature over their **own** intent.

The griefing sequence

1. Victim signs a legitimate Blaze intent with UUID `U` (e.g. a subnet transfer or an intent-DEX order). In an intent/solver system this signed payload — UUID and all — is visible off-chain before it is mined.
2. Attacker constructs any intent of their own, over `{contract: <attacker-principal>, intent: "...", opcode: none, amount: none, target: none, uuid: U}`, and signs it with the attacker's own key.
3. Attacker calls `blaze-v1.execute(sig_attacker, ..., uuid=U)` directly. `map-insert(U)` succeeds, `verify` recovers the attacker's own principal (a valid signature), `execute` returns `(ok attacker)`. `U` is now consumed globally. `blaze-v1.execute` performs no token movement itself, so the attacker's cost is one transaction's fee.
4. Victim's real intent (routed through its subnet contract, which calls `blaze-v1.execute` with the same `U`) now hits `map-insert(U) = false → ERR_UUID_SUBMITTED`. It reverts.

The victim is not robbed — they can re-sign with a new UUID and try again — but an attacker willing to spend gas can bounce a targeted victim's intents repeatedly, and because the map is shared, a UUID burned via a direct call to `blaze-v1` is unavailable to **every** subnet.

Live state (read-only verification)

The accompanying `verify_blaze_uuid_griefing.py` (python3 + curl, no wallet, no transaction) confirms against mainnet that:

- `blaze-v1` is deployed and its `submitted-uuids` map is declared `(string-ascii 36) -> bool` — keyed on the UUID alone.
- `execute` inserts the UUID before any signer-specific check, and the replay key is signer-independent.
- The public `check(uuid)` read-only function reflects the shared map, so already-consumed UUIDs read back `true` for anyone — demonstrating the single global namespace all subnets share.

No consuming transaction is sent; the attack step (4) is described, not executed, in keeping with responsible disclosure.

Impact

Denial of service against a targeted account's intents, protocol-wide. No theft, no fund loss, no permanent lock (the victim re-signs with a fresh UUID). Cost to the attacker is one transaction fee per grieved UUID plus a valid self-signed intent. This is likely regarded as partly inherent to nonce/UUID intent schemes; it is filed as a Low so the shared-namespaces aspect and a minimal hardening are on record.

Recommended fix

Bind the replay key to the intent's owner so unrelated parties cannot collide on a UUID:

- Key the replay map on `{ signer, uuid }` (or `{ contract-caller, uuid }`) instead of `uuid` alone — recover the signer first, then insert `{ signer, uuid }`. A third party signing their own intent then consumes only *their own* `{ attacker, U }` slot and cannot touch the victim's `{ victim, U }` slot.
- Alternatively, if UUIDs are meant to be globally unique per issuance, document that they must be unpredictable (high-entropy) and never revealed before on-chain submission, so they cannot be pre-empted. This is weaker than binding the key and does not help the mempool/solver-broadcast case.

Reward

Delivered as a good-faith private disclosure. Severity is Low (griefing DoS, no theft); no reward is expected, though consideration under any discretionary policy is appreciated. The priority is putting the shared-namespace behaviour and the one-line key change on record.